

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

NAME: E.V.Mahendra reddy

REGISTER NUMBER: 192425214

COURSE OUTCOME COVERED

***CO1:** Analyse reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr. G. A. Senthil
QUESTIONS: 10

Total Marks: 50

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyse the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

EXPERIMENT-1

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

```
C:\Users\rupas>python --version
Python 3.10.11

C:\Users\rupas>pip --version
pip 20.3 from C:\Users\rupas\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.10_qbz5n2kfra8p0\LocalCache\local-packages\Python310\site-packages\pip (python 3.10)

C:\Users\rupas>pip install numpy
Requirement already satisfied: numpy in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (2.2.6)
[notice] A new release of pip is available: 20.3 -> 20.3.2
[notice] To update, run: C:\Users\rupas\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.10_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

C:\Users\rupas>pip install matplotlib
Requirement already satisfied: matplotlib in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (3.8.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from matplotlib) (1.3.2)
Requirement already satisfied: cycler>=0.10 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.22 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from matplotlib) (2.2.6)
Requirement already satisfied: packaging>=20.0 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from matplotlib) (12.0.0)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from python-dateutil>=2.7->matplotlib) (1.19.0)
[notice] A new release of pip is available: 20.3 -> 20.3.2
[notice] To update, run: C:\Users\rupas\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.10_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

C:\Users\rupas>pip install gymnasium
Collecting gymnasium
  Downloading gymnasium-1.3.0-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: numpy>=1.21.0 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from gymnasium) (2.2.6)
Collecting cloudpickle>=1.2.0 (from gymnasium)
  Downloading cloudpickle-3.1.1-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: typing-extensions>=4.3.0 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from gymnasium) (4.15.0)
Collecting farama-notifications>=0.8.1 (from gymnasium)
  Downloading farama-notifications-0.8.0-py3-none-any.whl.metadata (720 bytes)
  Downloading farama-notifications-0.8.0-py3-none-any.whl (953 kB)
    4.1 MB 2.9 MB/s 0:00:00
  Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
  Downloading farama-notifications-0.8.0-py3-none-any.whl (2.9 kB)
Installing collected packages: farama-notifications, cloudpickle, gymnasium
Successfully installed cloudpickle-3.1.1 farama-notifications-0.8.0 gymnasium-1.3.0
    4.1 MB 2.9 MB/s 0:00:00

[notice] A new release of pip is available: 20.3 -> 20.3.2
[notice] To update, run: C:\Users\rupas\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.10_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

C:\Users\rupas>pip install gymnasium[classic-control]
Requirement already satisfied: gymnasium[classic-control] in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (1.3.0)
Requirement already satisfied: numpy>=1.21.0 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from gymnasium[classic-control]) (2.2.6)
Requirement already satisfied: cloudpickle>=1.2.0 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from gymnasium[classic-control]) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from gymnasium[classic-control]) (4.15.0)
Requirement already satisfied: farama-notifications>=0.8.1 in c:\users\rupas\appdata\local\packages\pythonsoftwarefoundation.python.3.10_qbz5n2kfra8p0\localcache\local-packages\python310\site-packages (from gymnasium[classic-control]) (0.8.0)
Collecting pygame-ce>=2.1.2 (from gymnasium[classic-control])
  Downloading pygame-ce-2.5.7-cp310-cp310-win_and64.whl.metadata (11 kB)
  Downloading pygame-ce-2.5.7-cp310-cp310-win_and64.whl (16.4 MB)
    16.4 MB 1.3 MB/s 0:00:00
Installing collected packages: pygame-ce
Successfully installed pygame-ce-2.5.7
    16.4 MB 1.3 MB/s 0:00:00

[notice] A new release of pip is available: 20.3 -> 20.3.2
[notice] To update, run: C:\Users\rupas\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.10_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
```

```
File Edit Format Run Options Window Help

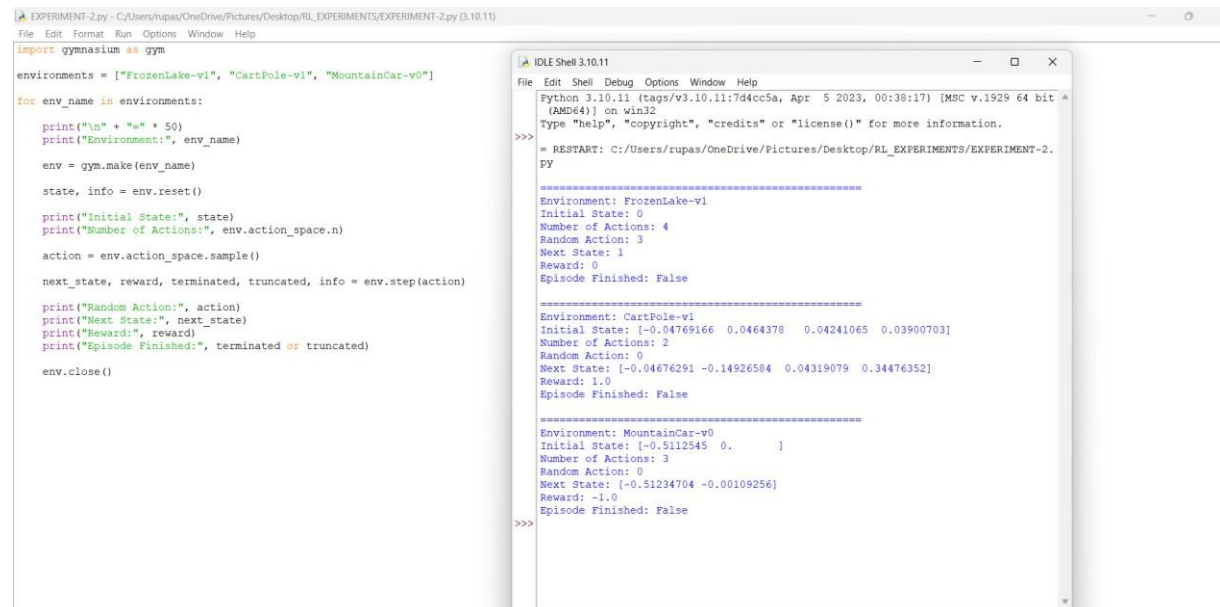
import numpy
import matplotlib
import gymnasium

print("NumPy Installed Successfully!")
print("Matplotlib Installed Successfully!")
print("Gymnasium Installed Successfully!")
print("Reinforcement Learning Environment Ready!")
```

```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 [tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17] [MSC v.1916 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information
>>>
= RESTART: C:\Users\rupas\OneDrive\Pictures\Desktop\RL_EXPERIMENTS\EX1
>>>
NumPy Installed Successfully!
Matplotlib Installed Successfully!
Gymnasium Installed Successfully!
Reinforcement Learning Environment Ready!
>>>
```

EXPERIMENT-2

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

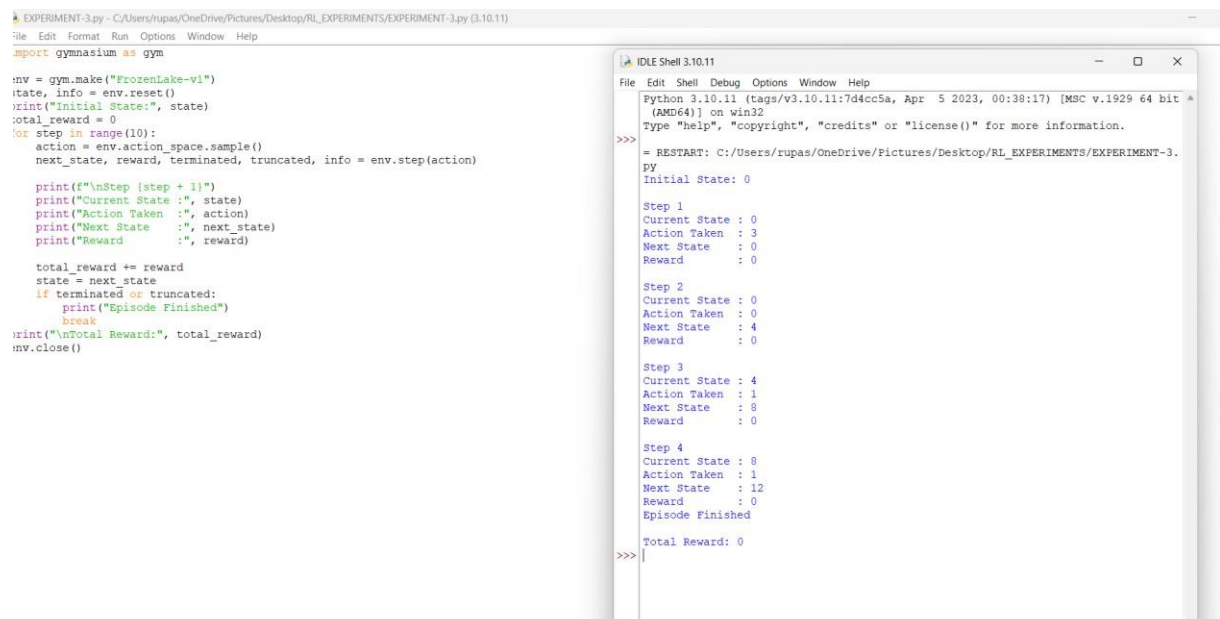


The screenshot shows a Python script in a text editor and its execution output in an IDLE Shell. The script defines three environments: FrozenLake-v1, CartPole-v1, and MountainCar-v0. It iterates through each environment, printing its name, initial state, number of actions, a random action, the next state, reward, and whether the episode finished. The output shows the following details for each environment:

- FrozenLake-v1:** Initial State: 0, Number of Actions: 4, Random Action: 3, Next State: 1, Reward: 0, Episode Finished: False.
- CartPole-v1:** Initial State: [-0.04769166 0.0464378 0.04241065 0.03900703], Number of Actions: 2, Random Action: 0, Next State: [-0.04676291 -0.14926584 0.04319079 0.34476352], Reward: 1.0, Episode Finished: False.
- MountainCar-v0:** Initial State: [-0.5112545 0.], Number of Actions: 3, Random Action: 0, Next State: [-0.51234704 -0.00109256], Reward: -1.0, Episode Finished: False.

EXPERIMENT-3

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.



The screenshot shows a Python script in a text editor and its execution output in an IDLE Shell. The script creates a FrozenLake-v1 environment and runs a loop for 10 steps. It prints the current state, action taken, next state, and reward at each step. The output shows the following details for each step:

- Step 1:** Current State: 0, Action Taken: 3, Next State: 0, Reward: 0.
- Step 2:** Current State: 0, Action Taken: 0, Next State: 4, Reward: 0.
- Step 3:** Current State: 4, Action Taken: 1, Next State: 8, Reward: 0.
- Step 4:** Current State: 8, Action Taken: 1, Next State: 12, Reward: 0.

The script ends with a print statement showing the total reward, which is 0.

EXPERIMENT-4

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

```
EXPERIMENT-4.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-4.py (3.10.11)
File Edit Format Run Options Window Help

states = ["S0", "S1", "S2", "Goal"]
rewards = {
    "S0": 0,
    "S1": 0,
    "S2": 0,
    "Goal": 10
}
transitions = {
    "S0": "S1",
    "S1": "S2",
    "S2": "Goal"
}
current_state = "S0"
total_reward = 0

print("Initial State:", current_state)

while current_state != "Goal":
    next_state = transitions[current_state]
    reward = rewards[next_state]

    print("-----")
    print("Current State :", current_state)
    print("Next State      :", next_state)
    print("Reward           :", reward)

    total_reward += reward
    current_state = next_state

print("-----")
print("Goal Reached!")
print("Total Reward:", total_reward)
```

```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [M
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more info
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMEN
py
Initial State: S0
-----
Current State : S0
Next State    : S1
Reward        : 0
-----
Current State : S1
Next State    : S2
Reward        : 0
-----
Current State : S2
Next State    : Goal
Reward        : 10
-----
Goal Reached!
Total Reward: 10
>>>
```

EXPERIMENT-5

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyse the convergence of value estimation.

```
EXPERIMENT-5.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-5.py (3.10.11)
File Edit Format Run Options Window Help

gamma = 0.9
rewards = {
    "S0": 0,
    "S1": 5,
    "S2": 10
}
values = {
    "S0": 0,
    "S1": 0,
    "S2": 0
}

print("Initial State Values")
for state in values:
    print(state, "=", values[state])

print("\nApplying Bellman Equation...\n")

values["S2"] = rewards["S2"]
values["S1"] = rewards["S1"] + gamma * values["S2"]
values["S0"] = rewards["S0"] + gamma * values["S1"]

print("Updated State Values")
for state in values:
    print(state, "=", round(values[state], 2))
best_state = max(values, key=values.get)

print("\nOptimal State:", best_state)
print("Maximum Value:", round(values[best_state], 2))
```

```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-5.
py
Initial State Values
S0 = 0
S1 = 0
S2 = 0

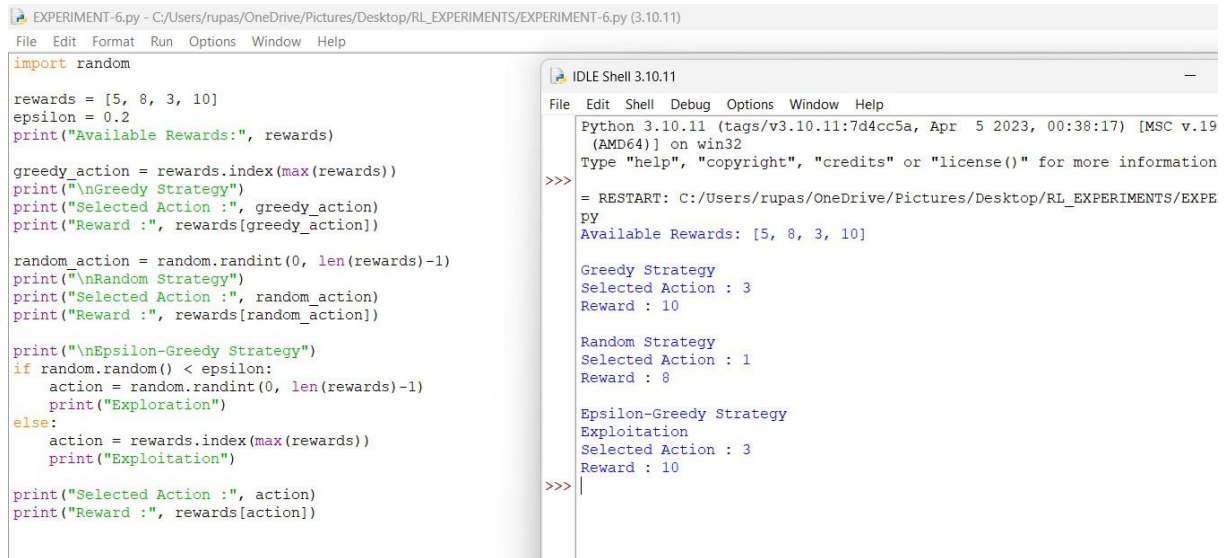
Applying Bellman Equation...

Updated State Values
S0 = 12.6
S1 = 14.0
S2 = 10

Optimal State: S1
Maximum Value: 14.0
>>>
```

EXPERIMENT-6

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.



```
EXPERIMENT-6.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-6.py (3.10.11)
File Edit Format Run Options Window Help

import random

rewards = [5, 8, 3, 10]
epsilon = 0.2
print("Available Rewards:", rewards)

greedy_action = rewards.index(max(rewards))
print("\nGreedy Strategy")
print("Selected Action :", greedy_action)
print("Reward :", rewards[greedy_action])

random_action = random.randint(0, len(rewards)-1)
print("\nRandom Strategy")
print("Selected Action :", random_action)
print("Reward :", rewards[random_action])

print("\nEpsilon-Greedy Strategy")
if random.random() < epsilon:
    action = random.randint(0, len(rewards)-1)
    print("Exploration")
else:
    action = rewards.index(max(rewards))
    print("Exploitation")

print("Selected Action :", action)
print("Reward :", rewards[action])

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.19
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPE
PY
Available Rewards: [5, 8, 3, 10]

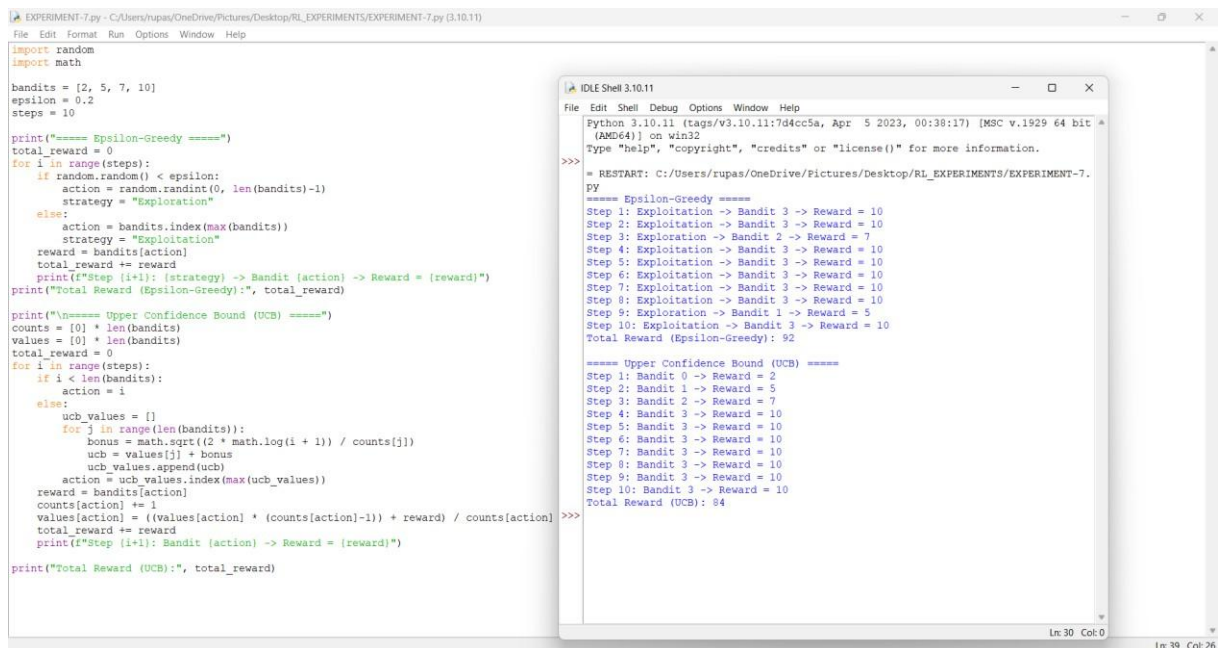
Greedy Strategy
Selected Action : 3
Reward : 10

Random Strategy
Selected Action : 1
Reward : 8

Epsilon-Greedy Strategy
Exploitation
Selected Action : 3
Reward : 10
>>>
```

EXPERIMENT-7

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.



```
EXPERIMENT-7.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-7.py (3.10.11)
File Edit Format Run Options Window Help

import random
import math

bandits = [2, 5, 7, 10]
epsilon = 0.2
steps = 10

print("==== Epsilon-Greedy =====")
total_reward = 0
for i in range(steps):
    if random.random() < epsilon:
        action = random.randint(0, len(bandits)-1)
        strategy = "Exploration"
    else:
        action = bandits.index(max(bandits))
        strategy = "Exploitation"
    reward = bandits[action]
    total_reward += reward
    print(f"Step {i+1}: {strategy} -> Bandit {action} -> Reward = {reward}")
print(f"Total Reward (Epsilon-Greedy):", total_reward)

print("\n==== Upper Confidence Bound (UCB) =====")
counts = [0] * len(bandits)
values = [0] * len(bandits)
total_reward = 0
for i in range(steps):
    if i < len(bandits):
        action = i
    else:
        ucb_values = []
        for j in range(len(bandits)):
            bonus = math.sqrt(2 * math.log(i + 1)) / counts[j]
            ucb = values[j] + bonus
            ucb_values.append(ucb)
        action = ucb_values.index(max(ucb_values))
    reward = bandits[action]
    counts[action] += 1
    values[action] = (values[action] * (counts[action]-1) + reward) / counts[action]
    total_reward += reward
    print(f"Step {i+1}: Bandit {action} -> Reward = {reward}")
print(f"Total Reward (UCB):", total_reward)

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-7.
PY
==== Epsilon-Greedy =====
Step 1: Exploitation -> Bandit 3 -> Reward = 10
Step 2: Exploitation -> Bandit 3 -> Reward = 10
Step 3: Exploration -> Bandit 2 -> Reward = 7
Step 4: Exploitation -> Bandit 3 -> Reward = 10
Step 5: Exploitation -> Bandit 3 -> Reward = 10
Step 6: Exploitation -> Bandit 3 -> Reward = 10
Step 7: Exploitation -> Bandit 3 -> Reward = 10
Step 8: Exploitation -> Bandit 3 -> Reward = 10
Step 9: Exploitation -> Bandit 3 -> Reward = 10
Step 10: Exploitation -> Bandit 1 -> Reward = 5
Total Reward (Epsilon-Greedy): 92

==== Upper Confidence Bound (UCB) =====
Step 1: Bandit 0 -> Reward = 2
Step 2: Bandit 1 -> Reward = 5
Step 3: Bandit 2 -> Reward = 7
Step 4: Bandit 3 -> Reward = 10
Step 5: Bandit 3 -> Reward = 10
Step 6: Bandit 3 -> Reward = 10
Step 7: Bandit 3 -> Reward = 10
Step 8: Bandit 3 -> Reward = 10
Step 9: Bandit 3 -> Reward = 10
Step 10: Bandit 3 -> Reward = 10
Total Reward (UCB): 84
>>>
```

Code of Conduct:

I E.V.Mahendra Reddy(192425214) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Mahendra Reddy

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand